TALLER DE CONSTRUCCIÓN DE SOFTWARE WEB

Gestión de estudiantes

Manejo de estado e integración de React con Spring Boot

Objetivo

Construir un registro de estudiantes que permita agregar, buscar y eliminar registros. La interfaz muestra el estado de las solicitudes y actualiza la lista tras una respuesta exitosa.

Cómo se integran los componentes

01 Interfaz React	App comparte la lista y los manejadores con Formulario y Directorio mediante props. useState guarda los registros, los campos del formulario y el texto de búsqueda.
02 Operaciones	El hook usarEstudiantes centraliza las solicitudes. useReducer administra las transiciones de carga, éxito y error de cada alta, baja o consulta. El filtro se calcula al escribir.
03 API Spring Boot	El controlador expone GET, POST y DELETE, valida los campos y evita códigos duplicados. Los datos de prueba se guardan en memoria mientras la API está activa.

Integrantes del grupo 7

Zafra Venegas, Andrey Hegel
Quispe Alvarado, Luis Guillermo
Ramirez Garcia, Estefano Alexis
Ramirez Alvarez, Maleck Adriano
Gutierrez Mamani, Joselin Victoria

Entorno de prueba

React en el puerto 5173 y Spring Boot en el puerto 8080. Las capturas de las páginas 2 a 4 corresponden a la ejecución local con la API real.

EVIDENCIA DE EJECUCIÓN

Directorio y búsqueda

React carga el listado desde Spring Boot y filtra los resultados mientras se escribe.

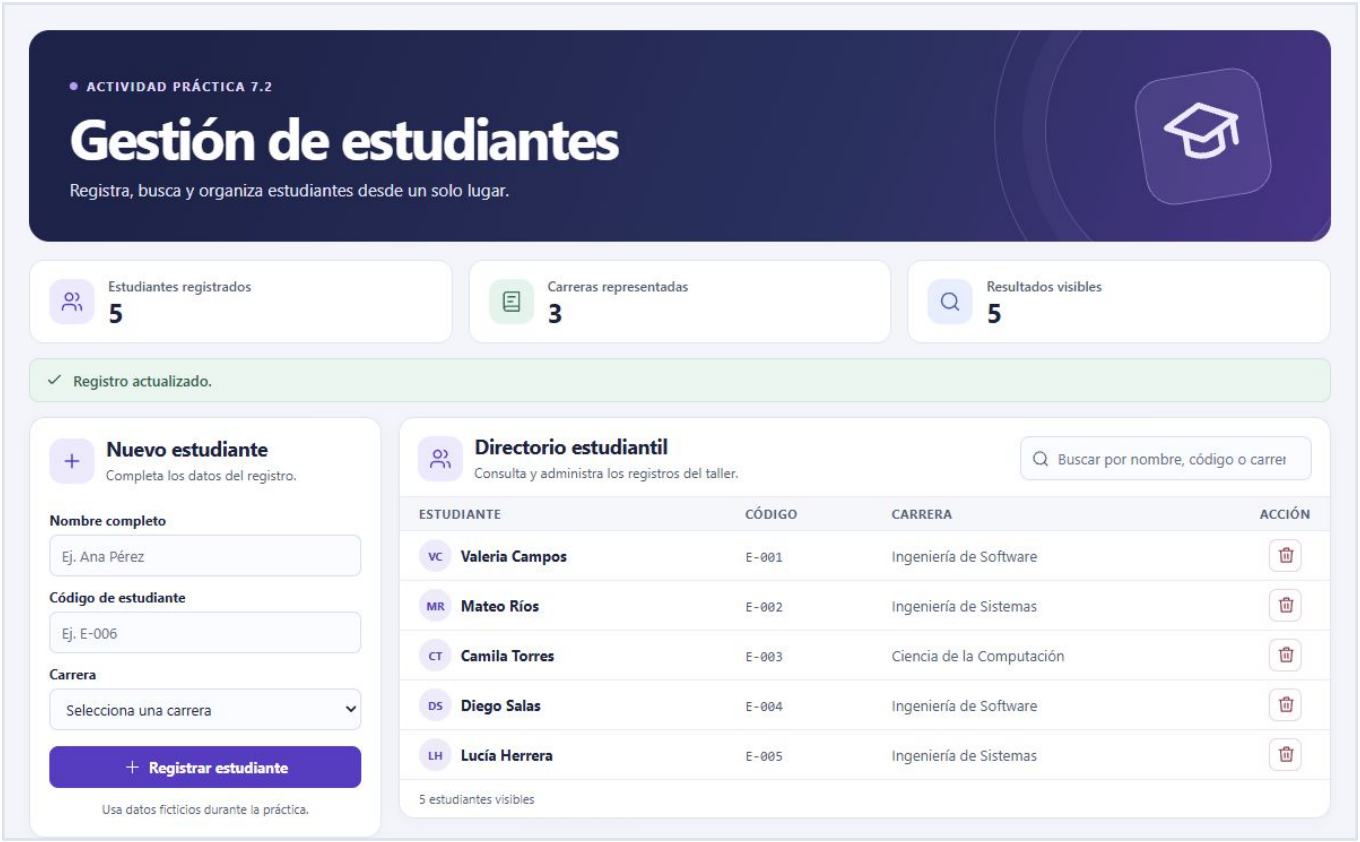


Figura 1. Carga inicial con la API activa

GET devuelve cinco registros ficticios distribuidos en tres carreras.

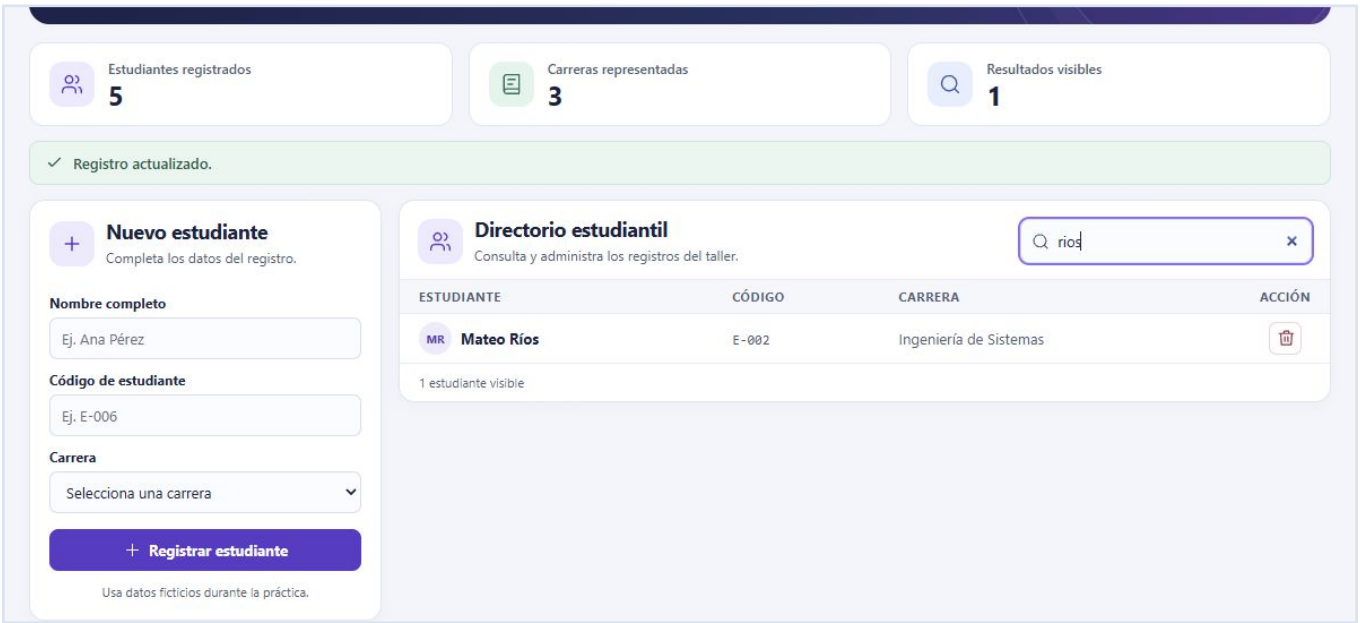


Figura 2. Filtro dinámico por nombre

Al escribir «rios», aparece Mateo Ríos. La búsqueda ignora tildes y mayúsculas.

EVIDENCIA DE EJECUCIÓN

Alta y baja de registros

El directorio y los contadores cambian después de que el servidor confirma cada operación.

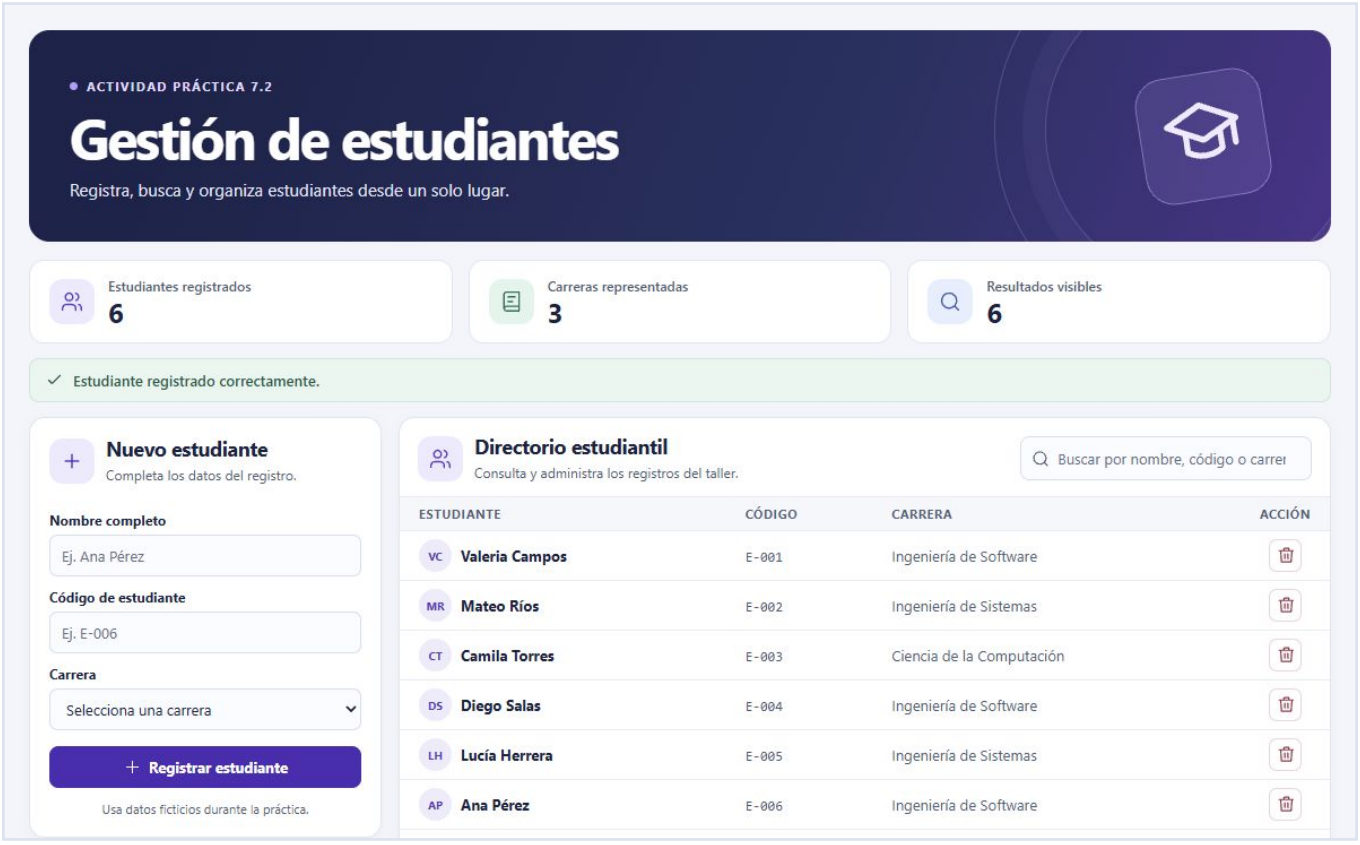


Figura 3. Alta confirmada por la API

POST devuelve 201. El registro E-006 aparece en el directorio y el total sube a seis.

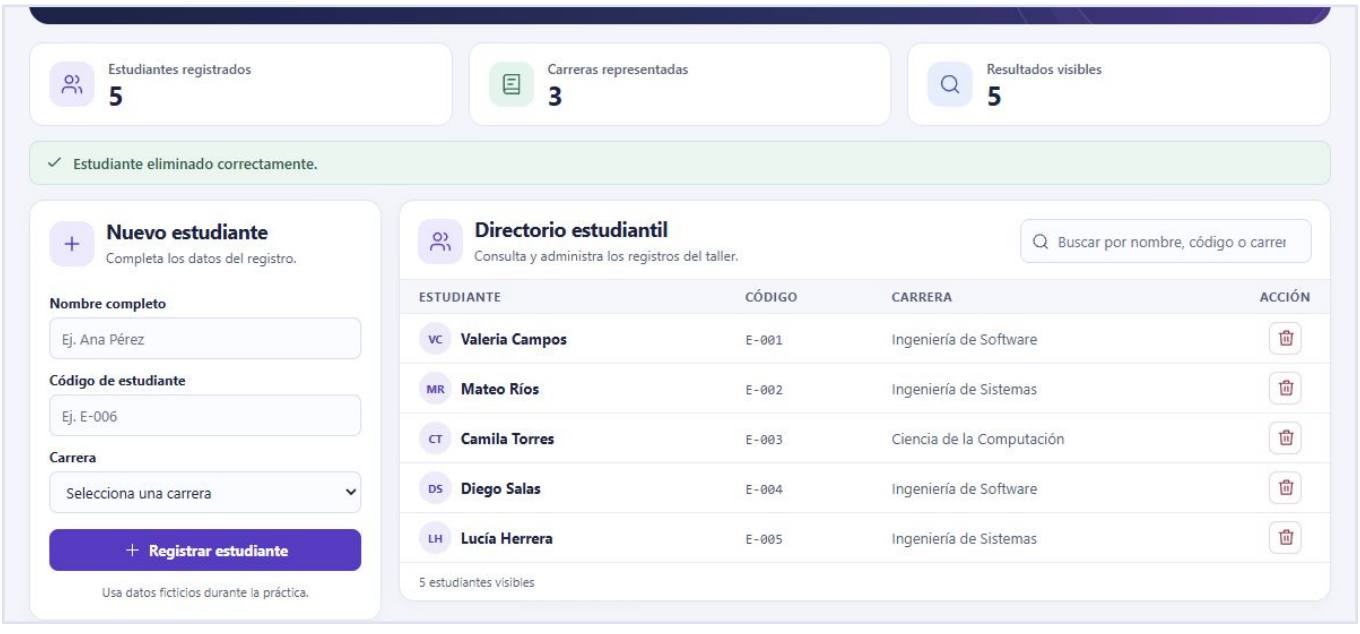


Figura 4. Baja confirmada por la API

Después de confirmar la eliminación de E-006, DELETE devuelve 204 y quedan cinco registros.

EVIDENCIA DE EJECUCIÓN

Errores y recuperación

Se probó el rechazo de códigos repetidos y la interrupción de la conexión con Spring Boot.

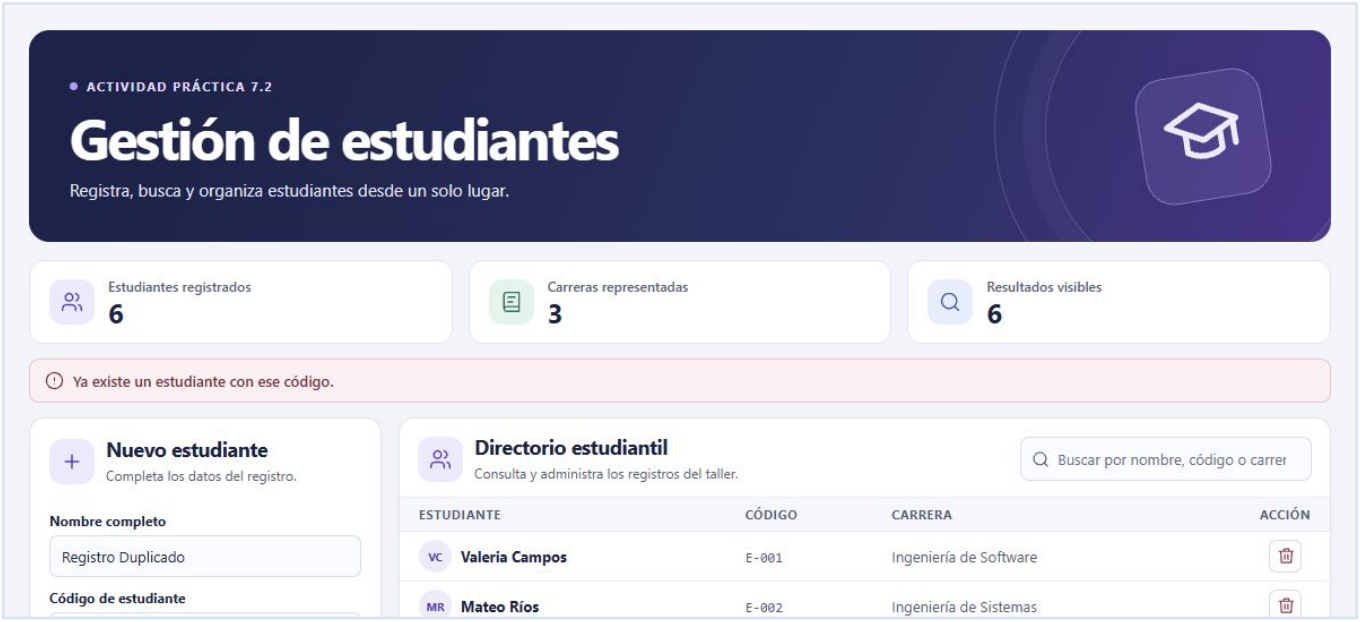


Figura 5. Código duplicado

El servidor responde con 409. La interfaz muestra el mensaje y conserva los seis registros.

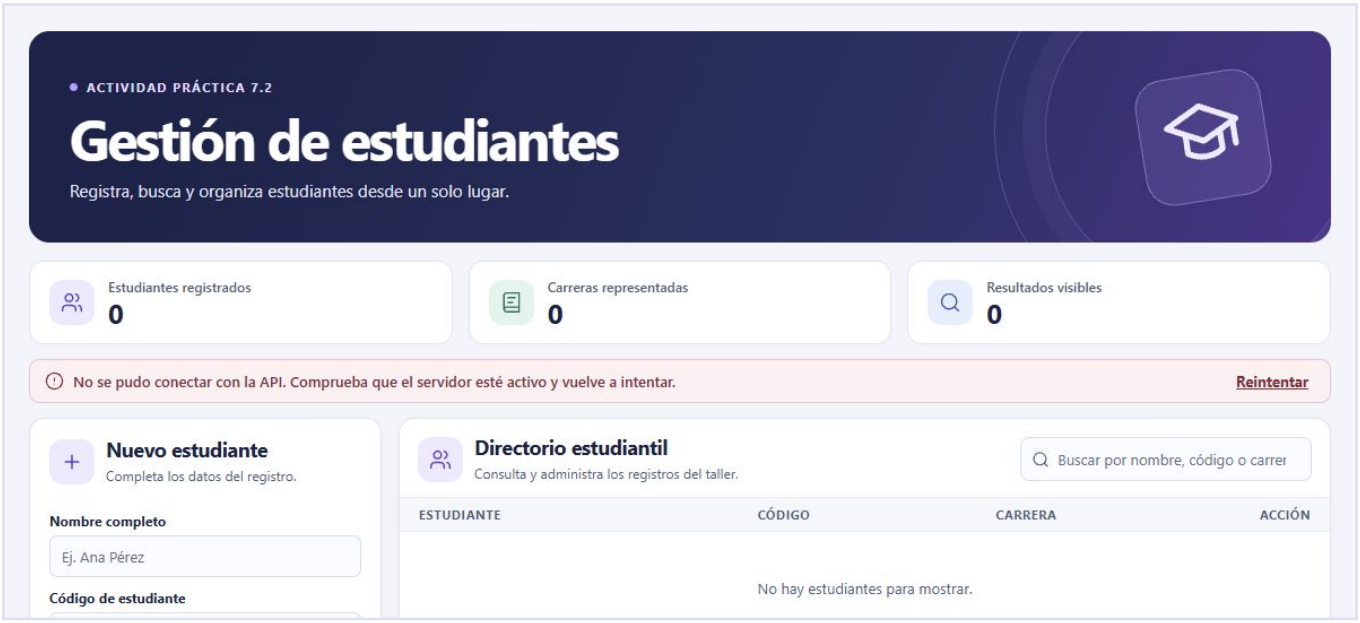


Figura 6. API detenida

La interfaz avisa que perdió la conexión. Al reiniciar Spring Boot y pulsar Reintentar, el directorio vuelve a cargar los cinco registros iniciales.

Respuestas comprobadas

GET 200 · POST 201, 400 y 409 · DELETE 204 y 404. También se comprobó que CORS admite el origen de GitHub Pages.

PREGUNTAS DE EVALUACIÓN

Estado en aplicaciones React

Conceptos aplicados a la implementación del registro estudiantil.

1. ¿Qué es el estado en frontend y por qué importa?

Es la información que cambia durante la ejecución y determina qué muestra la interfaz. Por ejemplo, al agregar un estudiante se vuelven a renderizar la lista y el contador.

2. ¿En qué se diferencian props y state?

Las props llegan al componente desde su padre y se leen como entradas. El state pertenece al componente que lo declara y se actualiza con el mecanismo que ofrece React.

3. ¿Para qué se utiliza useState?

Permite declarar un valor de estado y obtener una función para actualizarlo. En este proyecto se usa para la lista de estudiantes, los campos del formulario y la consulta de búsqueda.

4. ¿Qué significa elevar el estado?

Consiste en mover un estado al ancestro común más cercano de los componentes que lo comparten. Ese ancestro pasa valores y funciones por props para mantener sincronizada la vista.

5. ¿En qué se diferencian Context API y Redux Toolkit?

Context distribuye valores a componentes descendientes sin pasarlos por cada nivel. Redux Toolkit ofrece un almacén y convenciones para gestionar estado compartido con acciones, reducers y herramientas de depuración.

6. ¿Cuándo conviene usar useReducer?

Conviene cuando varias acciones producen transiciones relacionadas o cuando se quiere centralizar la lógica de actualización. Aquí organiza los resultados de las solicitudes de listado, alta y baja.

7. ¿En qué se diferencian el estado del cliente y el del servidor?

El cliente controla datos de interfaz como los campos y la búsqueda. El servidor administra los registros de estudiantes. React conserva una copia que sincroniza con las respuestas de la API.

Referencias técnicas

React · Manejo de estado
Redux Toolkit · Introducción
Spring · Construcción de un servicio REST